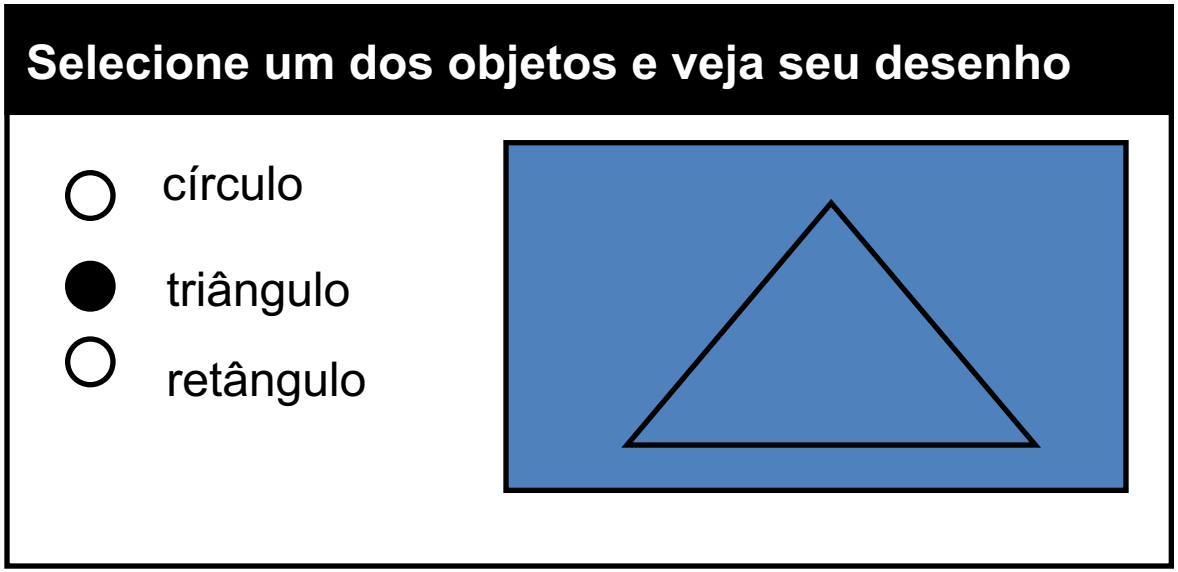
Padrões GoF – Visitor

Sabrina de F. Souto.

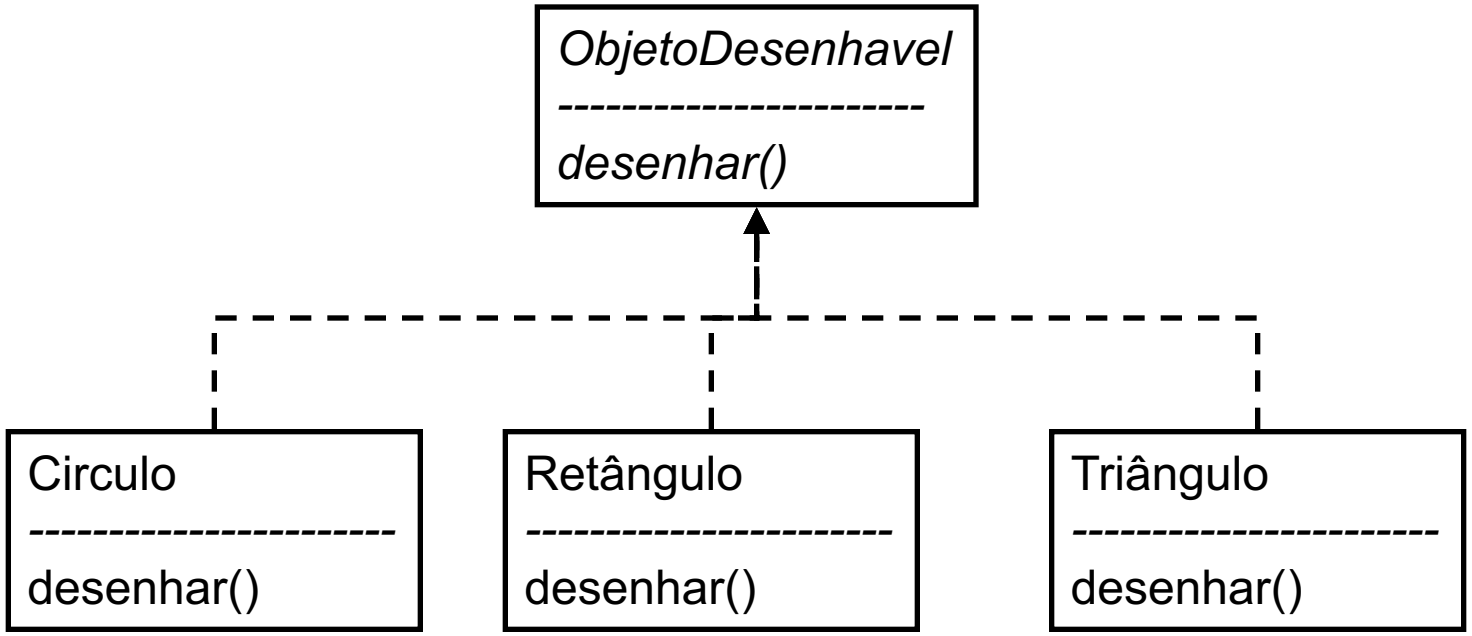
Figuras Geométricas

- Suponha que temos um sistema que desenha figuras geométricas de acordo com a escolha do usuário:



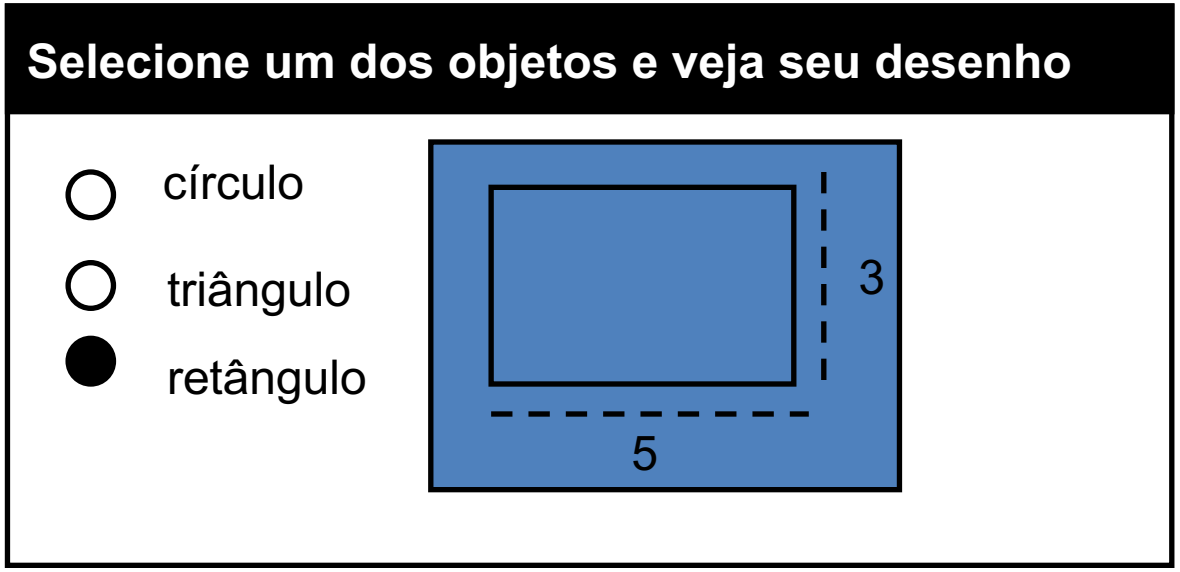
Figuras Geométricas

- Um diagrama para este exemplo seria:



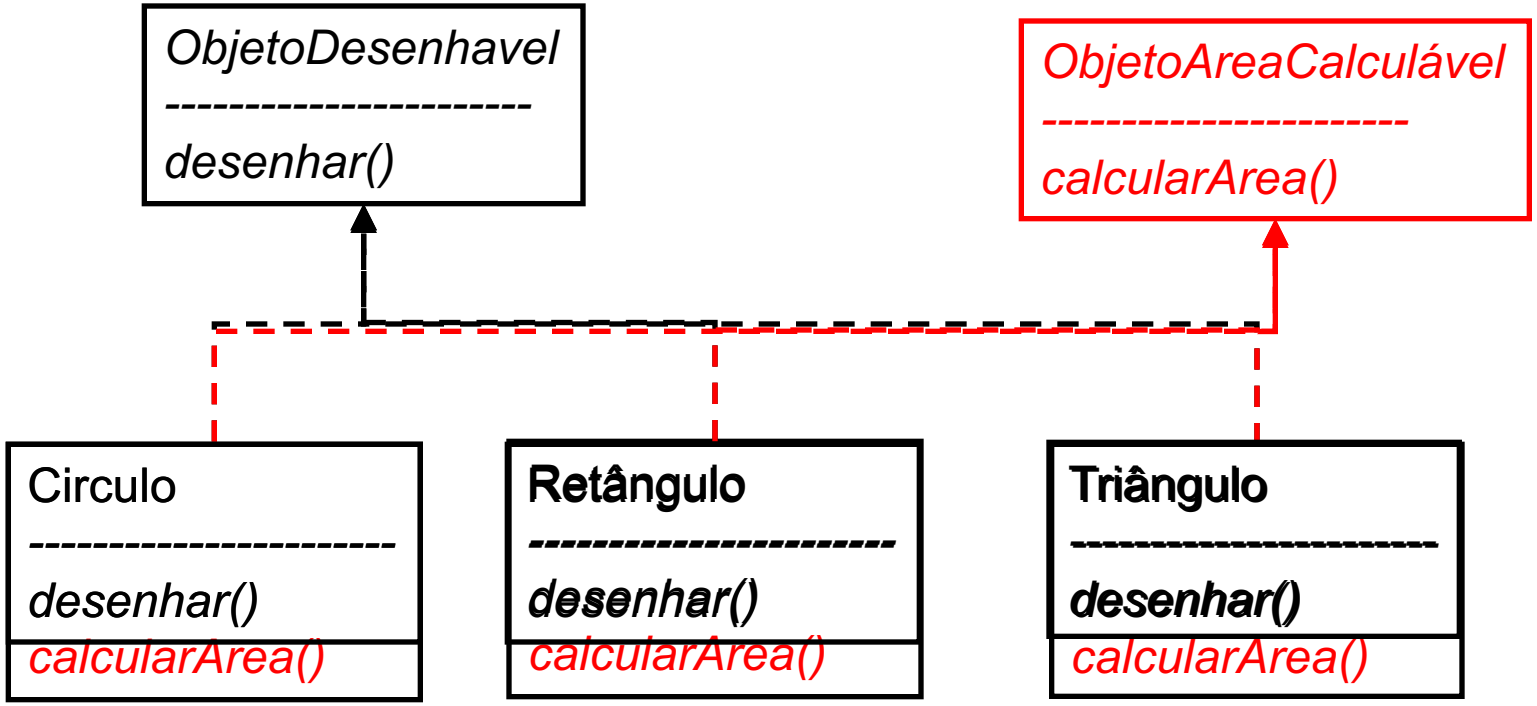
Figuras Geométricas

- E se agora a área do objeto selecionado também for exibida juntamente com o desenho?



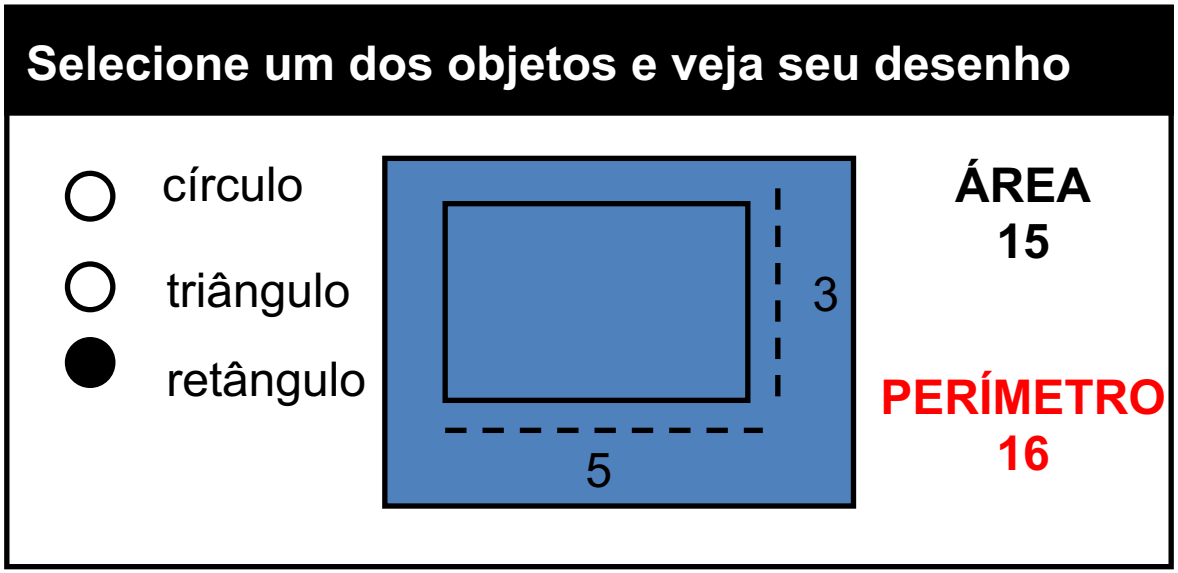
Figuras Geométricas

- O que muda em nosso diagrama???



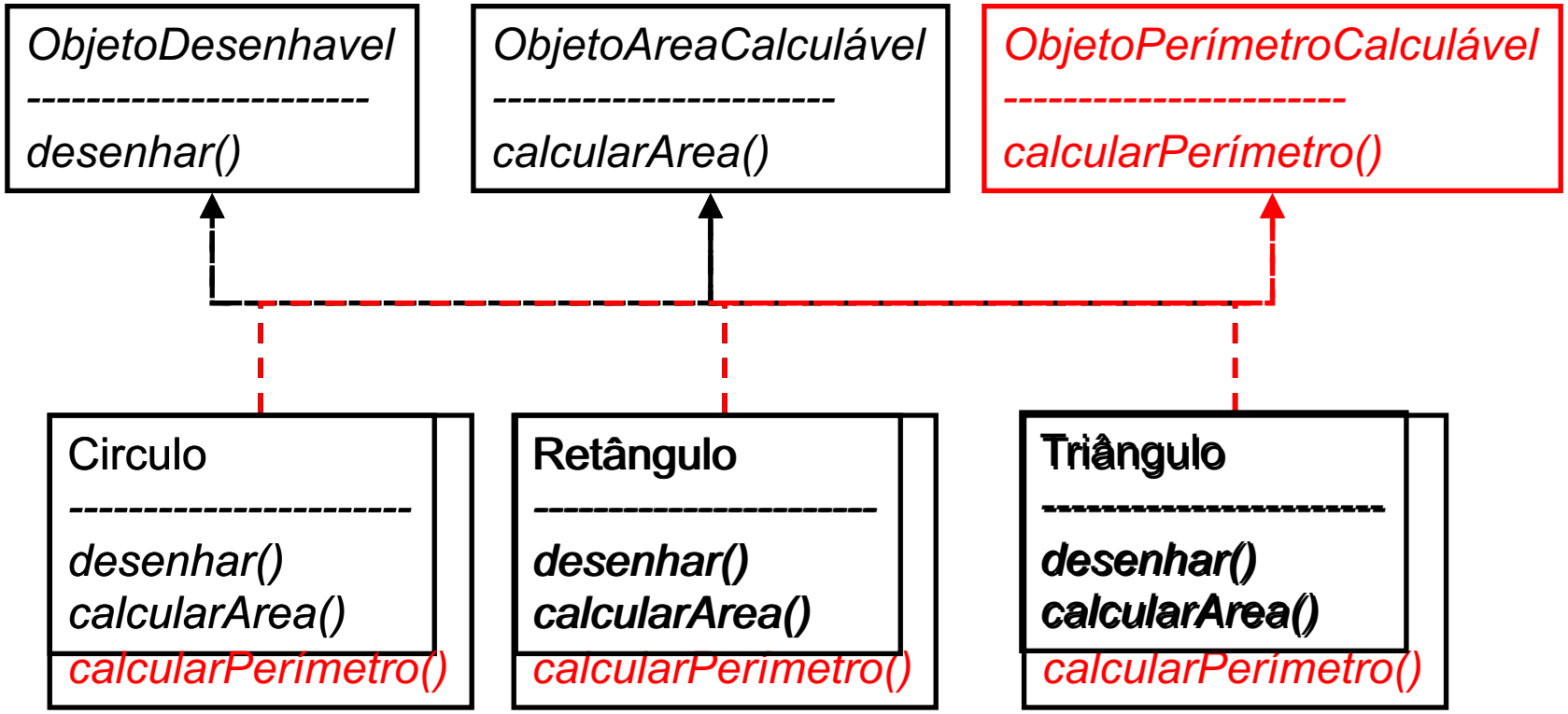
Figuras Geométricas

- E se, além do desenho exibirmos a área e o perímetro da figura geométrica visualizada?



Figuras Geométricas

- Nosso diagrama muda novamente!!!



Figuras Geométricas

- Quais as implicações destas mudanças?
  - Temos muitas operações polimórficas
    - As MESMAS operações aparecem espalhadas em DIFERENTES classes concretas
- Por que isto acontece?
  - As MESMAS operações são processadas de forma DIFERENTES para cada objeto geométrico
- Idéias para uma solução melhor?

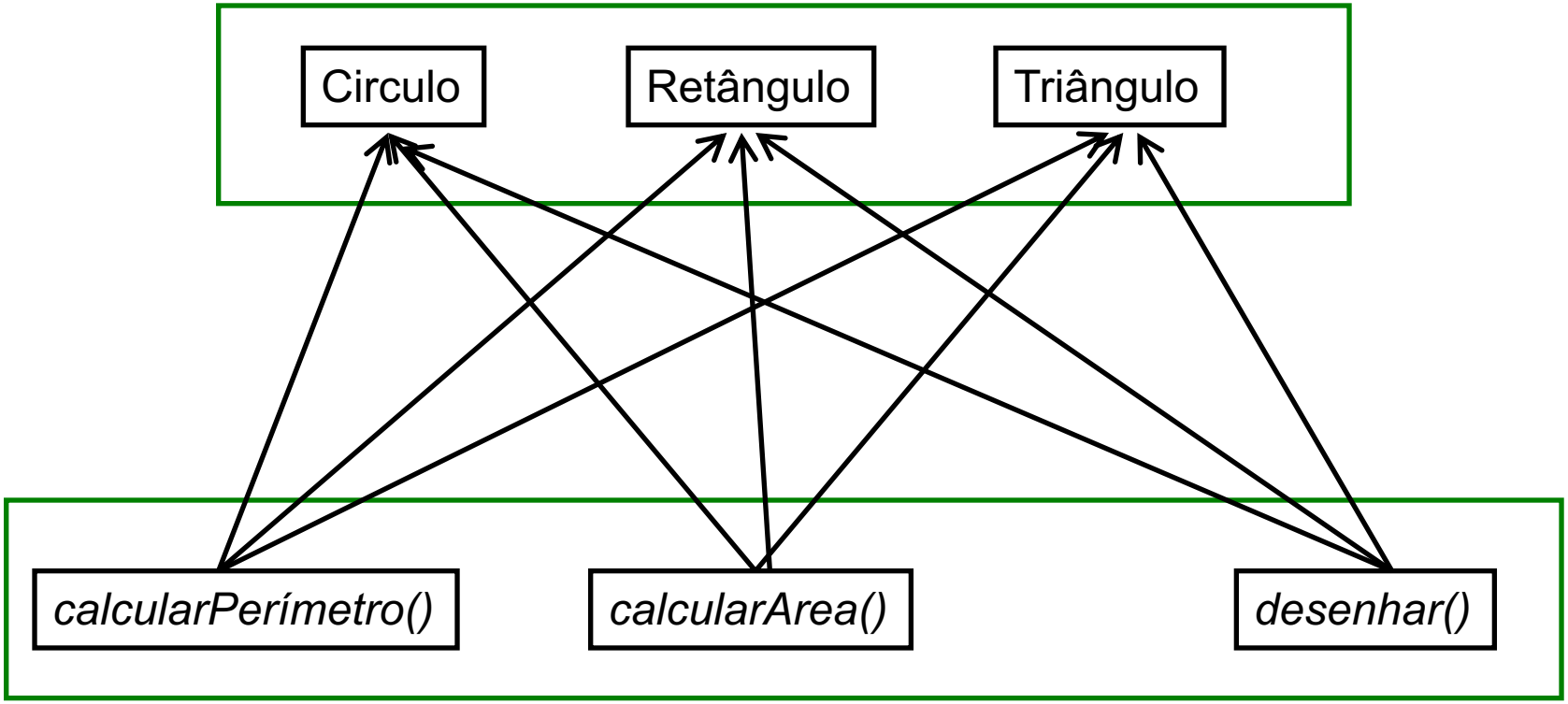
Figuras Geométricas

- Por que não separamos as **operações** dos **objetos** nos quais as operações são processadas?

- Como seria nosso diagrama para o mesmo exemplo das figuras geométricas considerando esta separação?

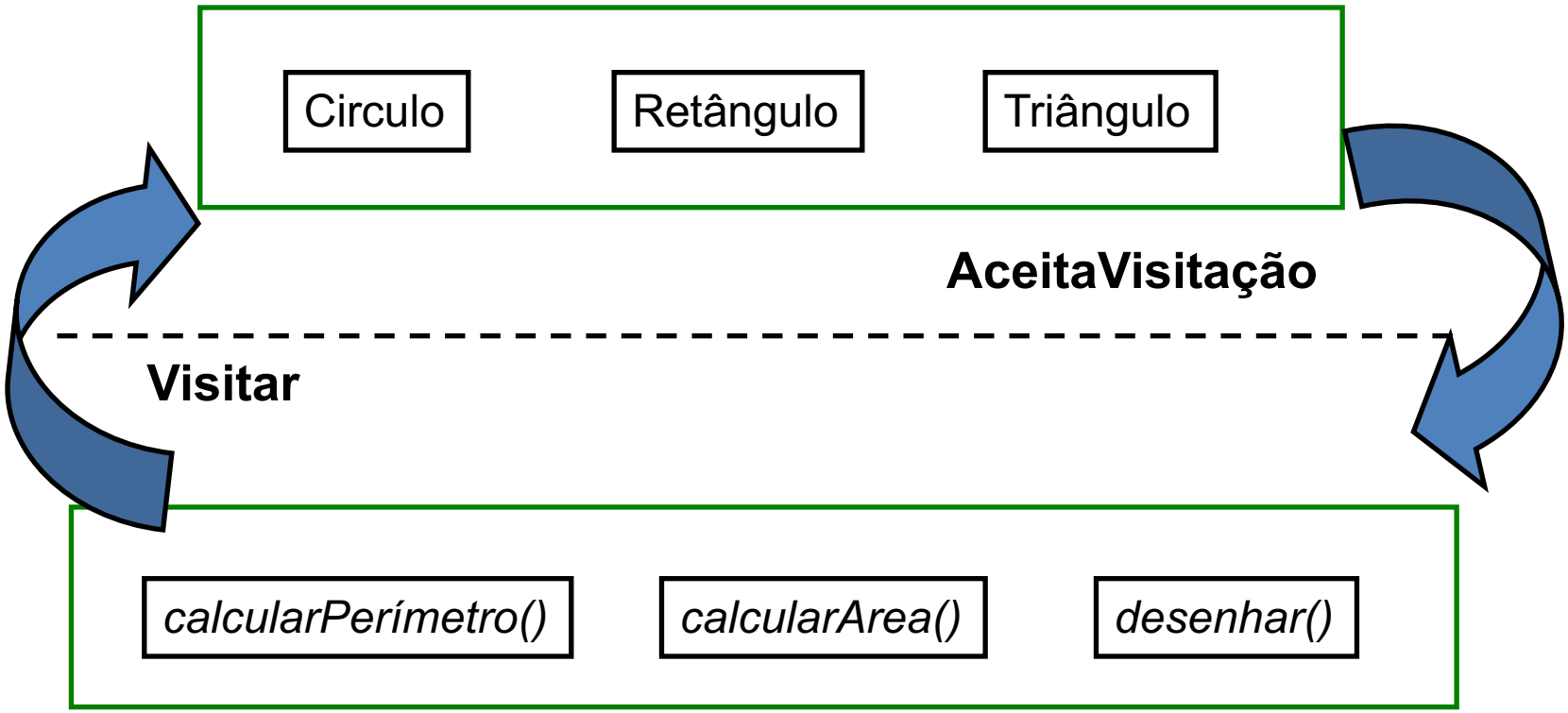
Figuras Geométricas

- Nova solução



Figuras Geométricas

- Como essas relações acontecem?



Padrão Visitor

Objetivo

- Representar uma operação a ser realizada nos elementos de uma estrutura de objetos distintos

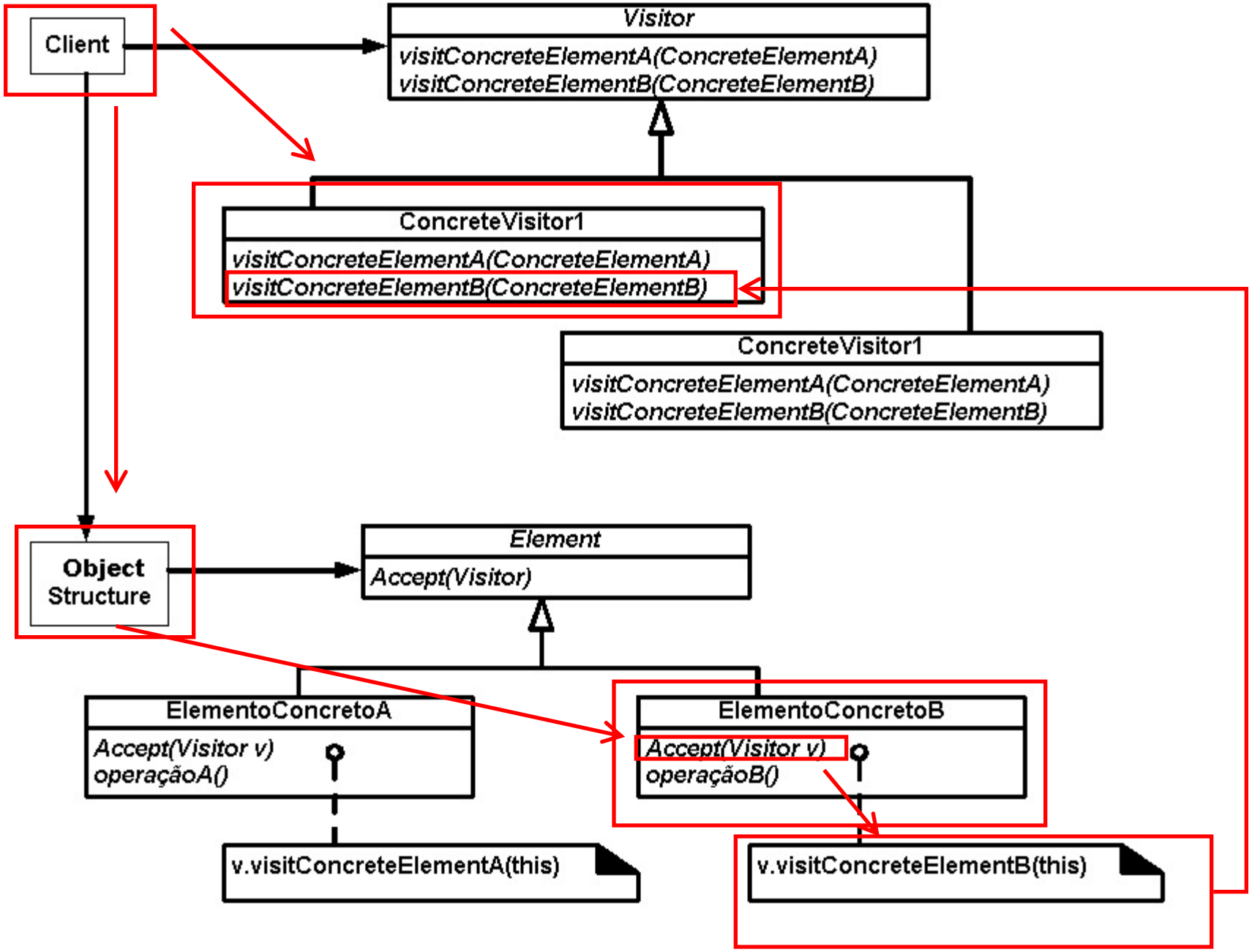
- Permitir que uma nova operação seja definida sem alterar as classes dos elementos nos quais a operação atua

Principais Participantes

- Visitante
  - Encapsula as operações
    - desenhar()
    - calcularArea()
    - calcularPerímetro()
- Elemento Concreto
  - Objetos nos quais as operações atuam
    - Circulo
    - Retângulo
    - Triângulo

Participantes

- Visitante
  - Declara uma operação de visita para cada classe de Elemento Concreto
- Elemento Concreto
  - Implementa uma operação aceitaVisita() que recebe um Visitante e chama a operação de visita apropriada deste Visitante


Client
Visitor
visitConcreteElementA(ConcreteElementA)
visitConcreteElementB(ConcreteElementB)
ConcreteVisitor1
visitConcreteElementA(ConcreteElementA)
visitConcreteElementB(ConcreteElementB)
ConcreteVisitor1
visitConcreteElementA(ConcreteElementA)
visitConcreteElementB(ConcreteElementB)
Object Structure
Element
Accept(Visitor)
ElementoConcretoA
Accept(Visitor v)
operaçãoA()
ElementoConcretoB
Accept(Visitor v)
operaçãoB()
v.visitConcreteElementA(this)
v.visitConcreteElementB(this)

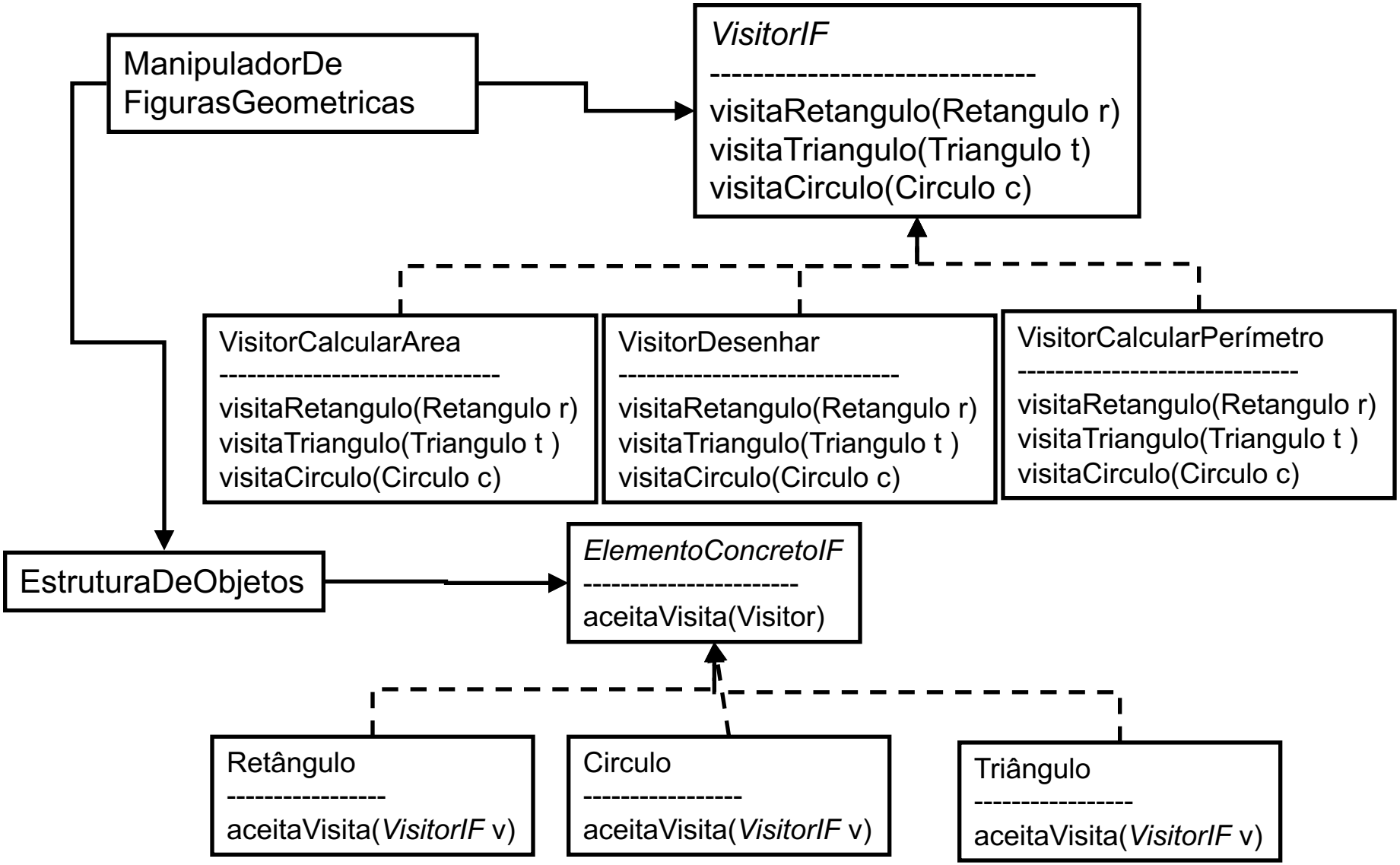
Colaborações

- O cliente deve criar um Visitor e visitar todos os elementos com este Visitor
- A estrutura de objetos chama accept(Visitor) para cada elemento
- Cada elemento chama visitConcreteElementXpto(this) do Visitor
  - Note que o Visitor recebe o elemento sendo visitado para poder acessá-lo se desejado

Resumo

- Cada operação está associada a um objeto Visitante
- Cada elemento aceita um objeto "visitante" a partir do método aceita(Visitante)
- O método aceita(Visitante) aciona os métodos do visitante
- A operação a ser feita dependerá tanto do tipo de Visitante quanto do tipo de elemento a ser visitado

Padrão Visitor para nosso Exemplo



Exemplo de Código

```
public interface Visitor IF {
   public double visitaRetangulo(Retangulo r);
   public double visitaTriangulo (Triangulo t);
   public double visitaCirculo (Circulo c);
}
```

Exemplo de Código

```
public class VisitorCalculaArea implements VisitorIF {

    public double visitaRetangulo(Retangulo r){
        return r.getAltura() * r.getLargura();
    }
    public double visitaTriangulo (Triangulo t) {
        return (t.getBase() * t.getAltura())/2;
    }
    public double visitaCirculo (Circulo c){
        return 2 * 3,14 * c.getRaio();
    }
}
```

VisitorCalculaPerimetro e VisitorDesenha são semelhantes

Exemplo de Código

```
public interface ElementoConcretoIF {

   public void aceitaVisita(VisitorIF v) ;
}
```

Exemplo de Código

```
public class Circulo implements ElementoConcretoIF {
    private double raio;

    public Circulo(double r) {
        this.raio = r;
    }

    public double getRaio() {
        return this.raio;
    }

    public void aceitaVisita(VisitorIF v) {
        v.visitaCirculo(this);
    }
}
```

Retangulo e Triangulo são semelhantes

Exemplo de Código

```
public class Principal {
   VisitorIF visitorCA = new VisitorCalculaArea();
   Circulo circulo = new Circulo(15);
   circulo.aceitaVisita(visitorCA);
}
```

```
visitorCA.visitaCirculo(this);
```

```
return 3,14 * (c.getRaio() * c.getRaio());
```

706,5

Consequências de uso

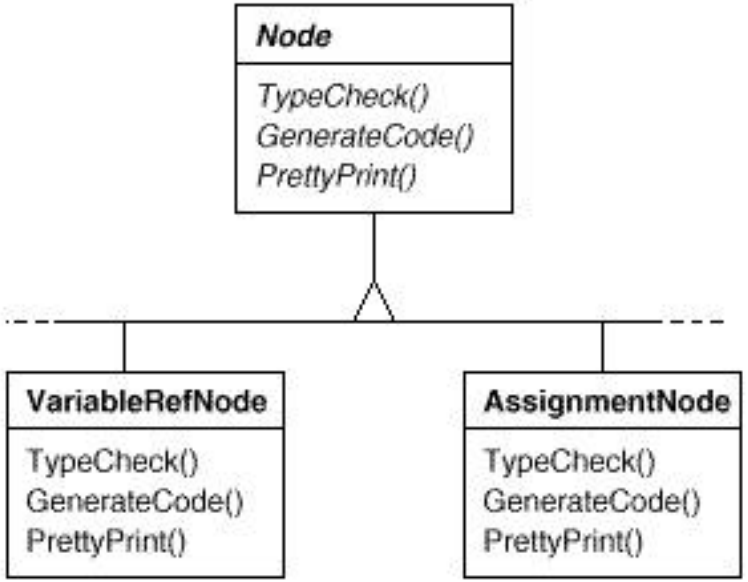
- Permite adicionar novas operações com facilidade
- Junta operações relacionadas e separa operação não relacionadas
- Adicionar novas classes ConcreteElement é difícil
- Permite visitas em hierarquias diferentes
- Quebra de encapsulação
  - Já que a operação está dentro de um Visitor o ConcreteElement poderá ter que expor mais interfaces do que seria desejável

Exemplo em Compiladores

- Representação de um programa
  - Árvore sintática abstrata
    - A árvore possui diferentes tipos de nós
      - » Operadores;
      - » Variáveis;
      - » Expressões matemáticas;
- Exemplo de operações de um compilador na árvore sintática abstrata
  - Formatação;
  - Verificação de tipos;
  - Verificar se todas as variáveis estão definidas;
  - Geração de código

Exemplo em Compiladores

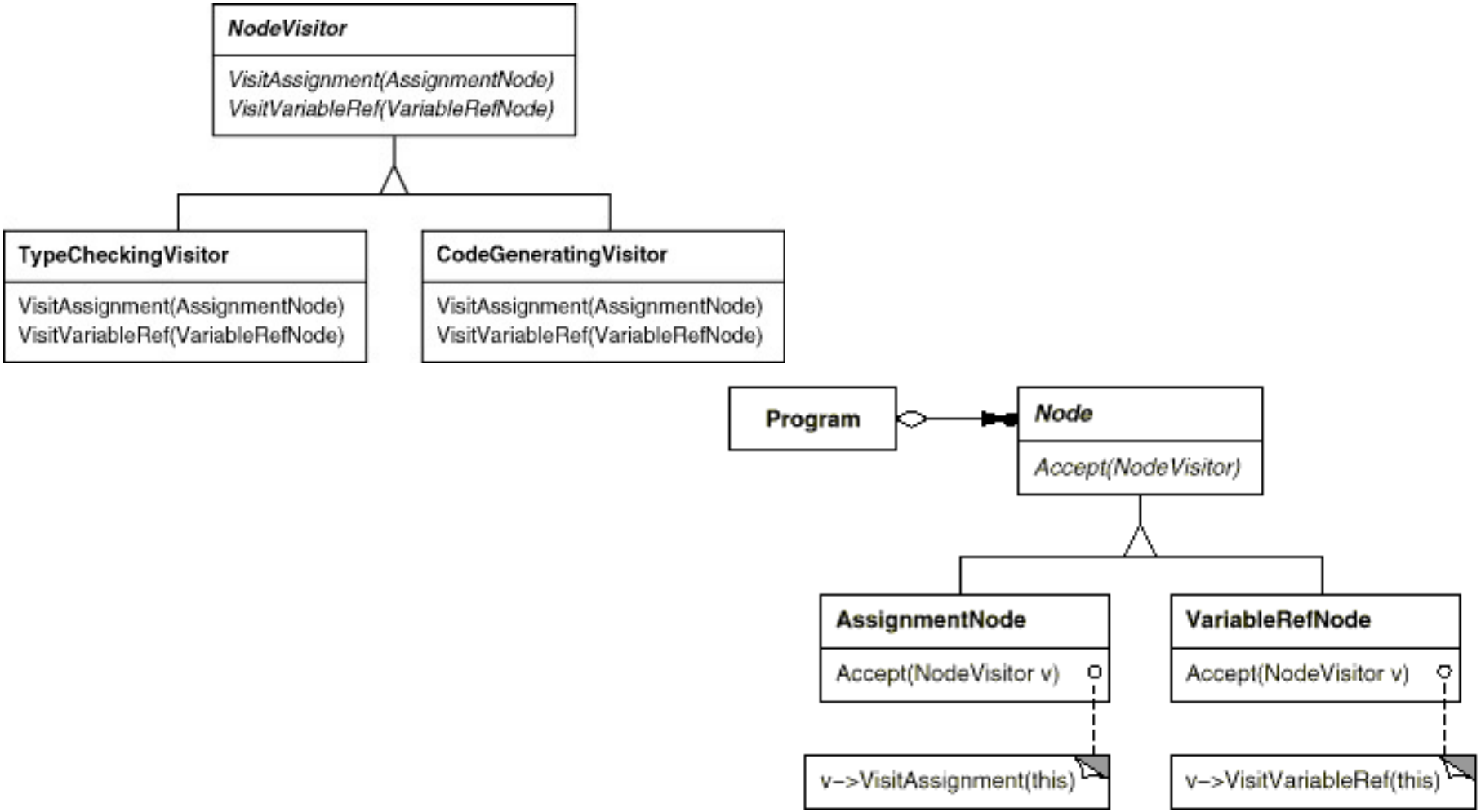
- **Problema**
  - As operações podem necessitar tratar cada tipo de nó de maneira diferente
- **Solução usual**
  - Definição da operação na classe que a utiliza;
  - Desvantagens
    - Inclusão de novas operações requer a mudança de todas as classes de nós;
    - Poluição das classes com muitas operações
      - Mistura de verificação de tipos com formatação



Exemplo em Compiladores

- **Solução por padrão**
  - Encapsular a operação em um objeto em separado (Visitor)
  - Algoritmo do Visitor
    - O objeto Visitor percorre os elementos da árvore;
    - Se um nó da árvore "aceita" um visitor então
      - chama um método seu que inclui o tipo do nó como argumento;
      - Executa a operação para aquele nó (a operação que costumava estar na classe nó)

Exemplo em Compiladores



Dúvida?

Laboratório: Visitor

Implemente o exemplo Figuras Geométricas visto em sala. Considere os objetos: Circulo, Triângulo, Retângulo e Trapézio. Considere as operações: desenhar, calcular áarea, calcular perímetro e maximizar figura (duplicar raio, lado, base, altura, etc.).

Itens avaliados:

- 30% - Elementos Concretos;
- 30% - Visitantes;
- 10% - Main, onde devem ser criados os visitantes e elementos concretos e realizar todas as visitas possíveis;
- 30% - Testes.